

ggbeeswarm README examples

p9beeswarm and ggbeeswarm side by side

This page recreates the [upstream ggbeeswarm README examples](#). The Python and upstream R plots are executed in adjacent Quarto cells and displayed side by side. Click **Code** to expand either implementation.

Rendering

Render from the repository root:

```
quarto render docs/ggbeeswarm-examples.qmd
```

Rendering requires R packages `ggplot2`, `ggbeeswarm`, and `reticulate`, plus the project Python environment created by `uv sync --no-default-groups --group test`.

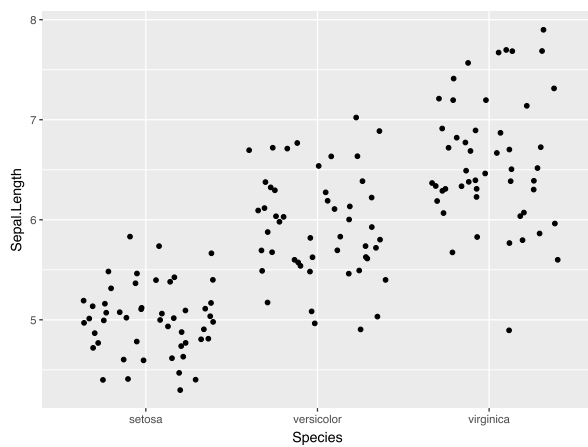
Jitter and quasirandom

```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + p9.geom_jitter()).show()
```

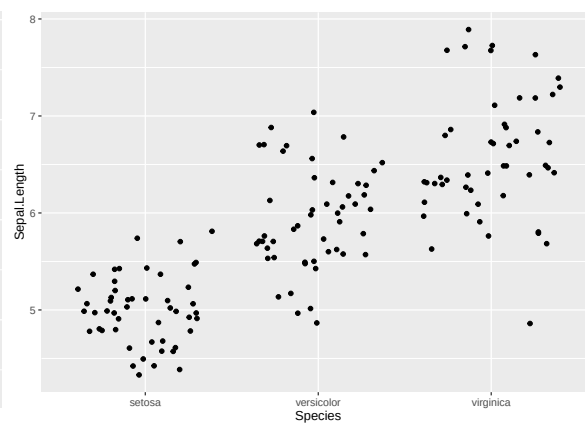
```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_jitter())
```

```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_quasirandom(random_state=12345)).show()
```

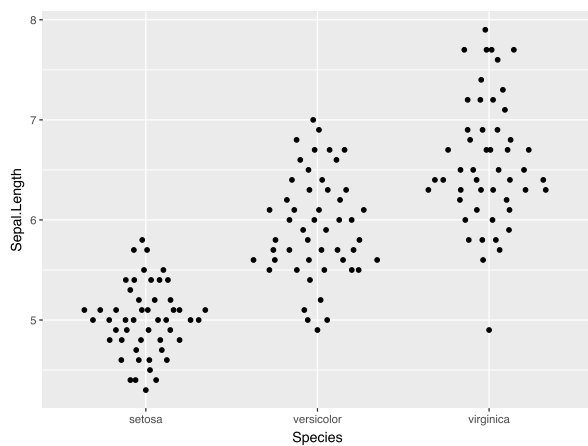
```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_quasirandom())
```



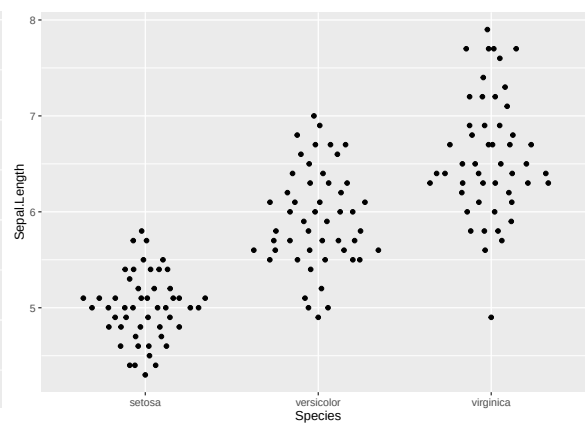
(a) Python: jitter



(a) R: jitter



(a) Python: quasirandom



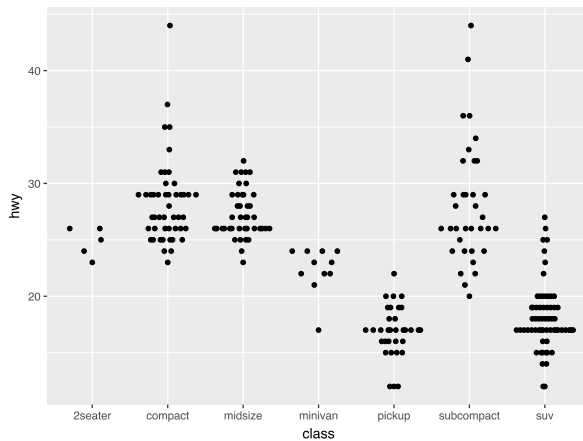
(a) R: quasirandom

geom_quasirandom

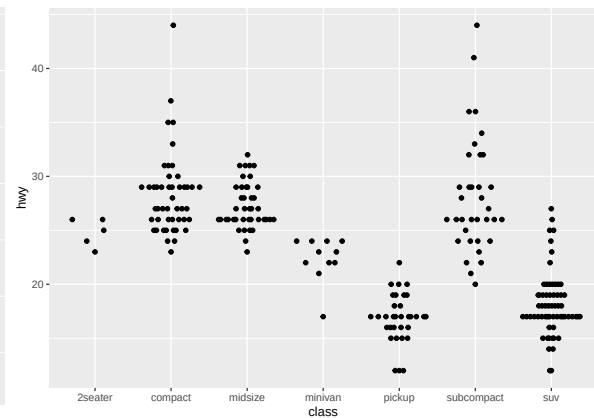
Basic usage

```
(p9.ggplot(mpg, p9.aes("class", "hwy")) + geom_quasirandom(random_state=12345)).show()
```

```
print(ggplot(mpg, aes(class, hwy)) + geom_quasirandom())
```



(a) Python: basic geom_quasirandom



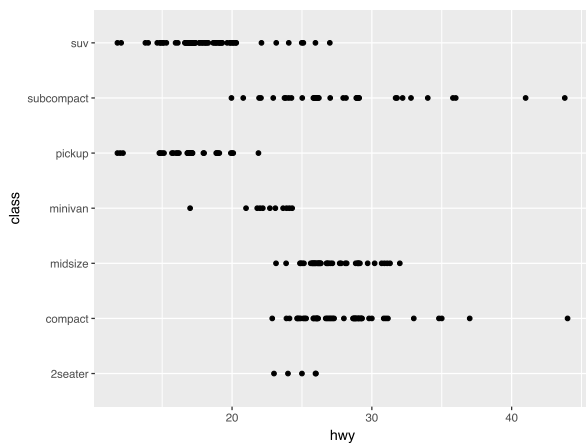
(a) R: basic geom_quasirandom

```
(p9.ggplot(mpg, p9.aes("hwy", "class")) +  
  geom_quasirandom(orientation = 'x', group_on_x=False, random_state=12345)).show()
```

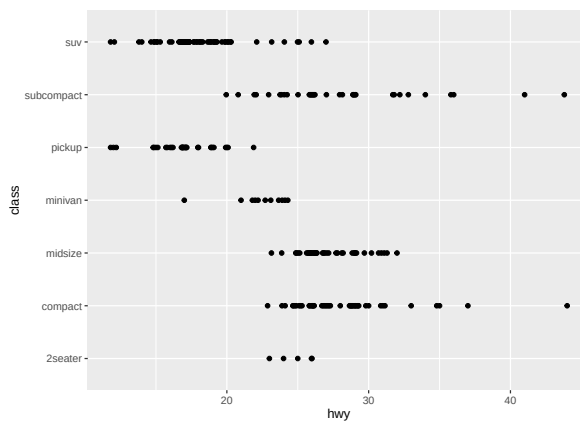
```
print(ggplot(mpg, aes(hwy, class)) +  
  geom_quasirandom(orientation = 'x', groupOnX = FALSE))
```

```
(p9.ggplot(mpg, p9.aes("class", "hwy")) +  
  geom_quasirandom(varwidth=True, random_state=12345)).show()
```

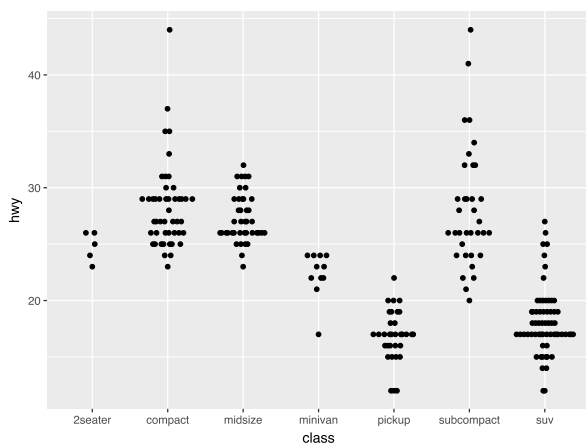
```
print(ggplot(mpg, aes(class, hwy)) +  
  geom_quasirandom(varwidth = TRUE))
```



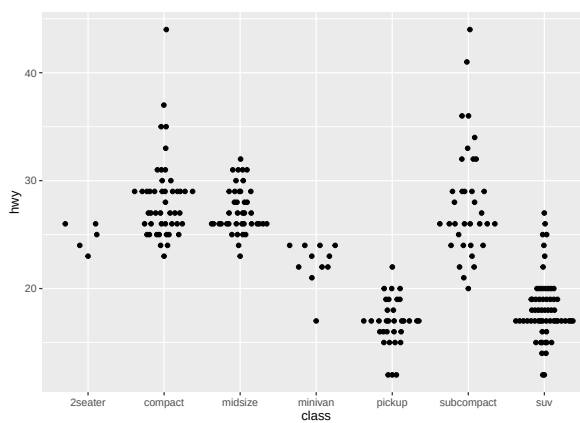
(a) Python: horizontal geom_quasirandom



(a) R: horizontal geom_quasirandom



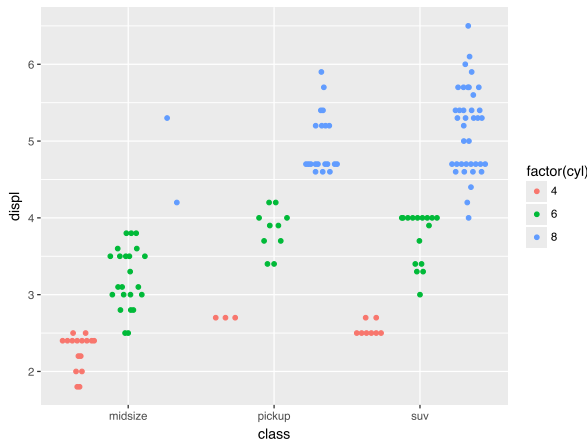
(a) Python: varwidth geom_quasirandom



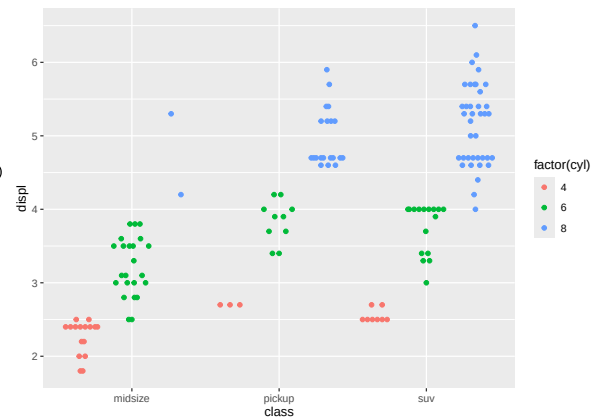
(a) R: varwidth geom_quasirandom

```
(p9.ggplot(sub_mpg, p9.aes("class", "displ", color="factor(cyl)")) +  
  geom_quasirandom(dodge_width=1, random_state=12345)).show()
```

```
print(ggplot(sub_mpg, aes(class, displ, color = factor(cyl))) +  
  geom_quasirandom(dodge.width = 1))
```



(a) Python: dodged geom_quasirandom



(a) R: dodged geom_quasirandom

Alternative methods

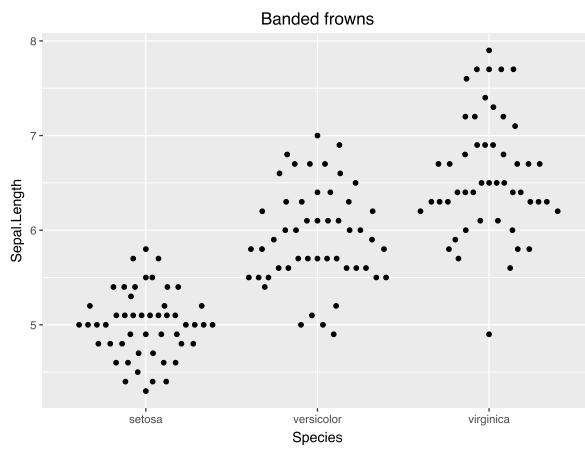
```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_quasirandom(method="frowney", random_state=12345)).show()
```

```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_quasirandom(method = "frowney") + ggtitle("Frowney"))
```

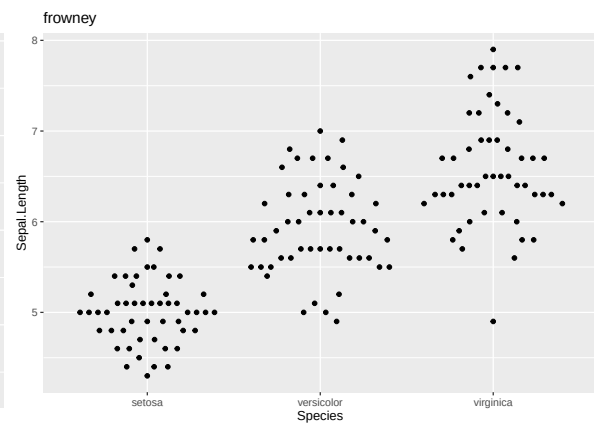
```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_quasirandom(method="smiley", random_state=12345)).show()
```

```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_quasirandom(method = "smiley") + ggtitle("Smiley"))
```

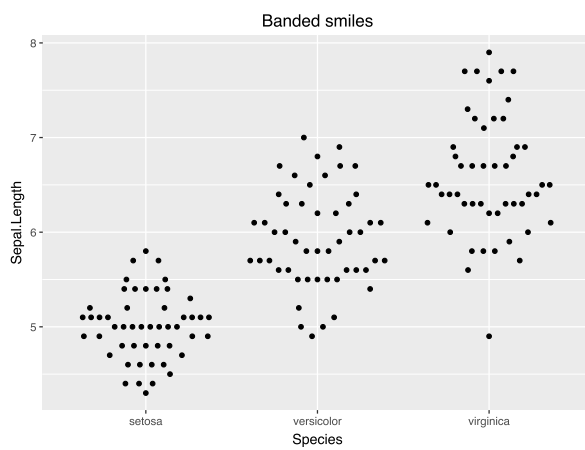
```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) +  
  geom_quasirandom(method="tukey", random_state=12345) +  
  p9.ggtitle("Tukey")).show()
```



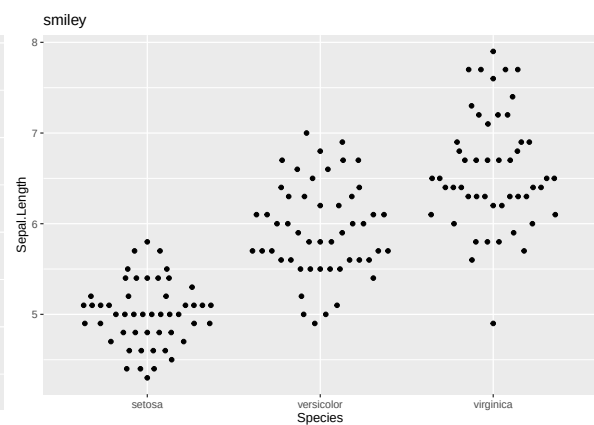
(a) Python: frowney quasirandom



(a) R: frowney quasirandom

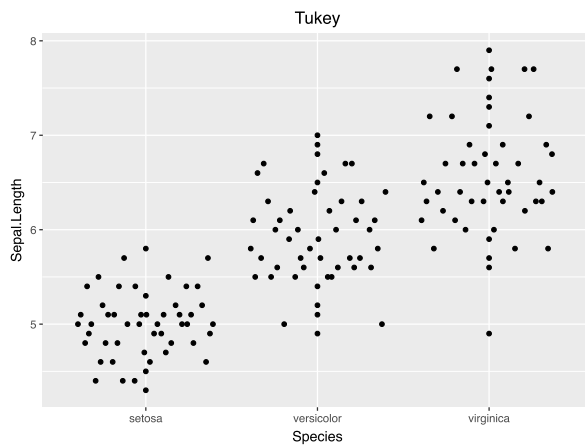


(a) Python: smiley quasirandom

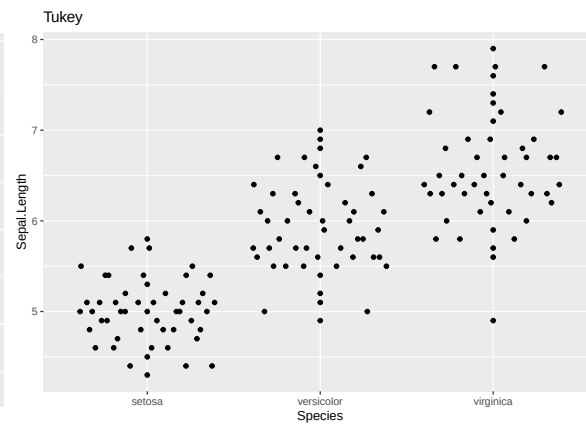


(a) R: smiley quasirandom

```
print(ggplot(iris, aes(Species, Sepal.Length)) +  
      geom_quasirandom(method = "tukey") +  
      ggtitle("Tukey"))
```



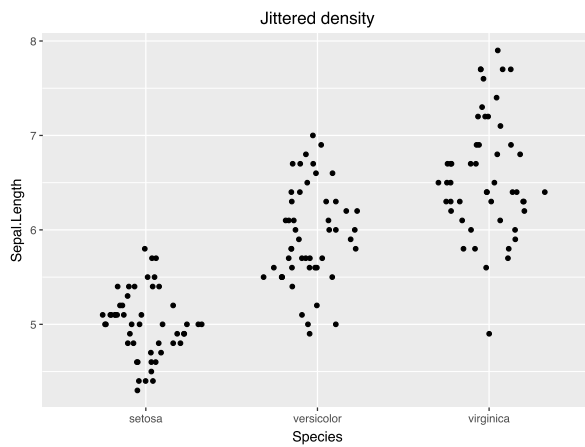
(a) Python: Tukey texture



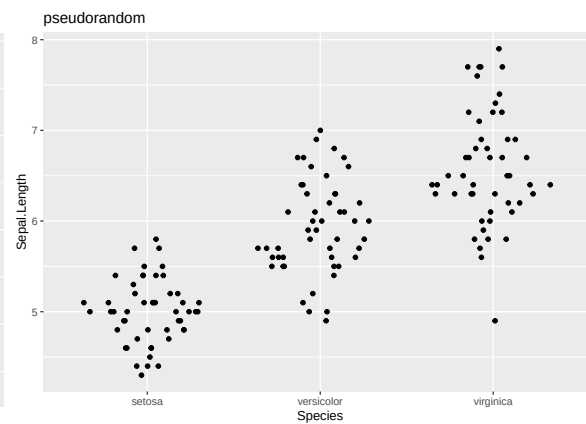
(a) R: Tukey texture

```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_quasirandom(method="pseudorandom"
```

```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_quasirandom(method = "pseudorandom") +
```



(a) Python: pseudorandom quasirandom



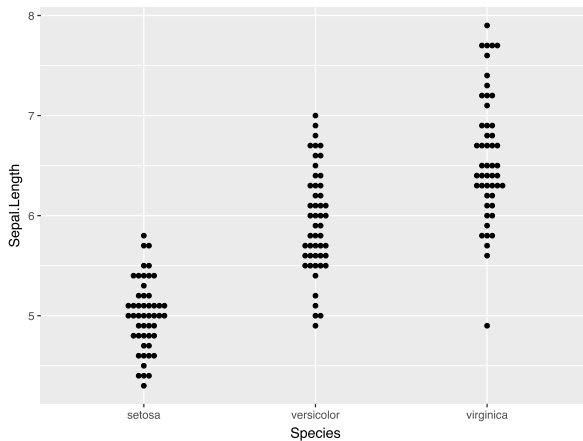
(a) R: pseudorandom quasirandom

geom_beeswarm

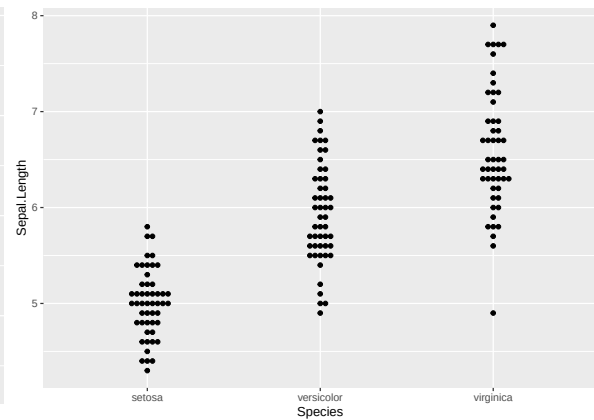
Basic usage

```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_beeswarm()).show()
```

```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_beeswarm())
```



(a) Python: basic geom_beeswarm



(a) R: basic geom_beeswarm

```
(p9.ggplot(iris, p9.aes("Species", "Sepal.Length")) + geom_beeswarm(side=1)).show()
```

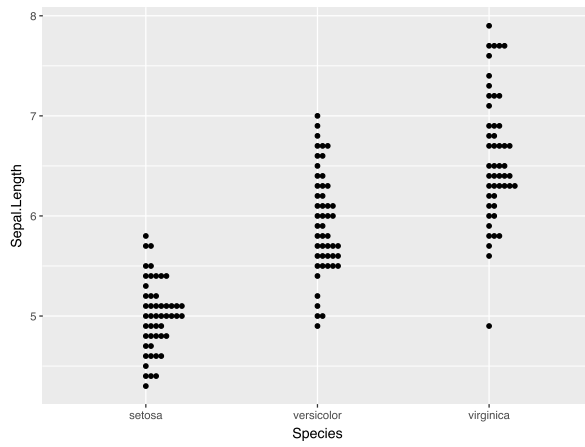
```
print(ggplot(iris, aes(Species, Sepal.Length)) + geom_beeswarm(side = 1L))
```

```
(p9.ggplot(mpg, p9.aes("class", "hwy")) + geom_beeswarm(size=0.5)).show()
```

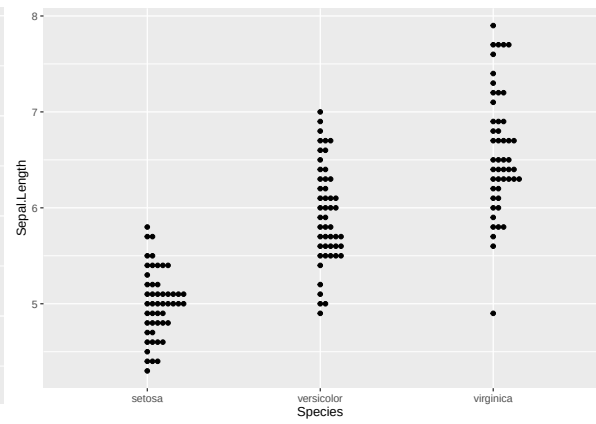
```
print(ggplot(mpg, aes(class, hwy)) + geom_beeswarm(size = .5))
```

```
(p9.ggplot(mpg, p9.aes("hwy", "class")) + geom_beeswarm(size=0.5)).show()
```

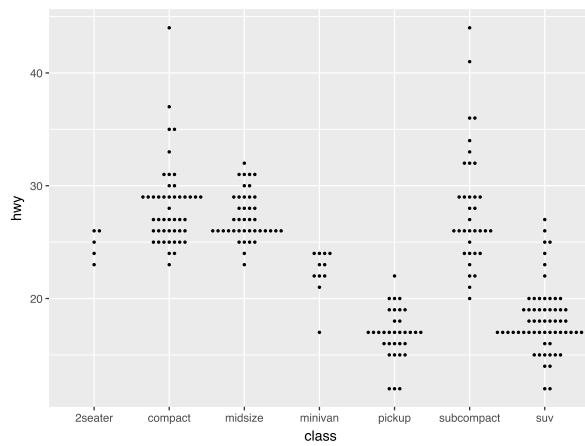
```
print(ggplot(mpg, aes(hwy, class)) + geom_beeswarm(size = .5, orientation='x'))
```



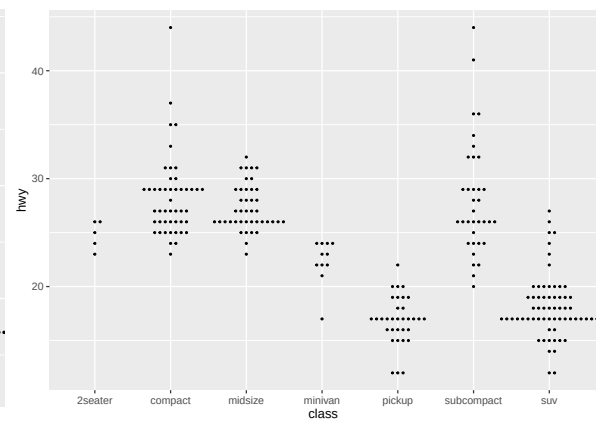
(a) Python: one-sided geom_beeswarm



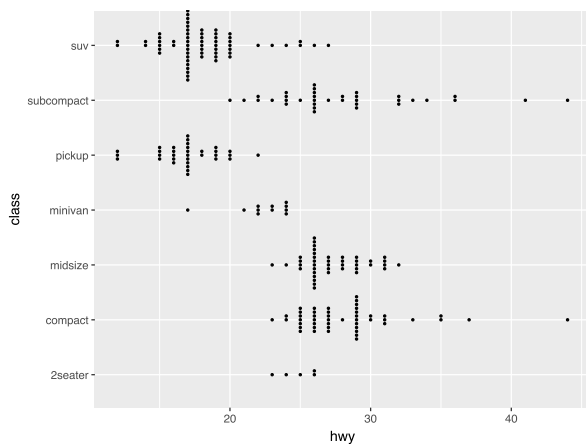
(a) R: one-sided geom_beeswarm



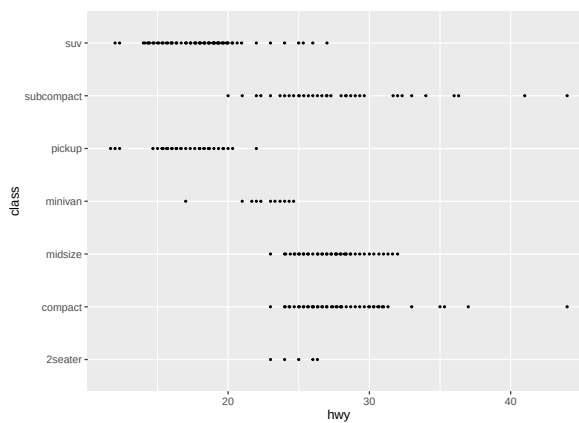
(a) Python: vertical geom_beeswarm



(a) R: vertical geom_beeswarm



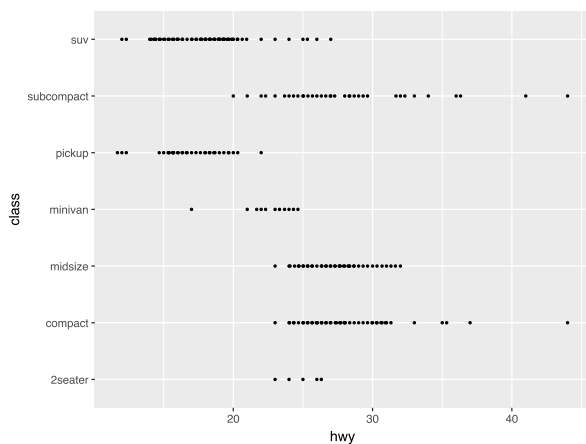
(a) Python: horizontal geom_beeswarm



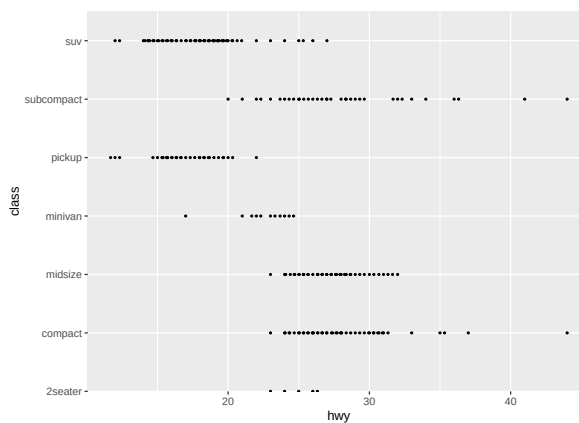
(a) R: horizontal geom_beeswarm

```
(p9.ggplot(mpg, p9.aes("hwy", "class")) +
  geom_beeswarm(size=0.5, orientation="x") +
  p9.scale_y_discrete(expand=(0, 0.5))).show()
```

```
print(ggplot(mpg, aes(hwy, class)) +
  geom_beeswarm(size = .5, orientation='x') +
  scale_y_discrete(expand = expansion(add = c(0, 0.5))))
```



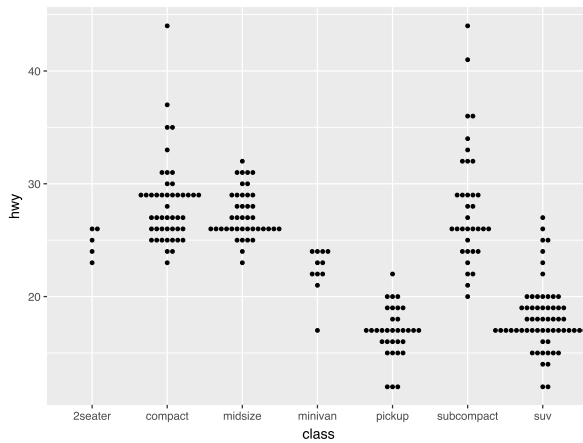
(a) Python: expanded horizontal geom_beeswarm



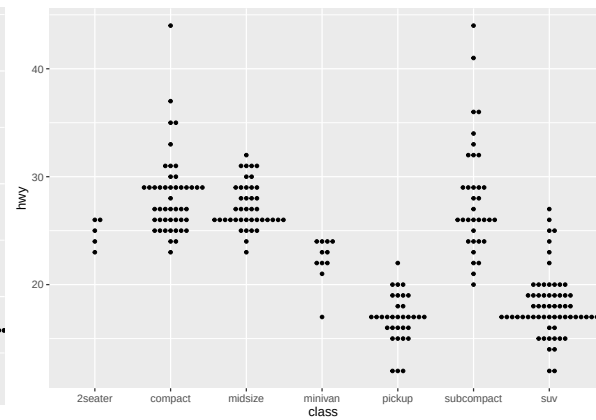
(a) R: expanded horizontal geom_beeswarm

```
(p9.ggplot(mpg, p9.aes("class", "hwy")) +
  geom_beeswarm(size=1.1)).show()
```

```
print(ggplot(mpg, aes(class, hwy)) +
  geom_beeswarm(size = 1.1))
```



(a) Python: large-point geom_beeswarm



(a) R: large-point geom_beeswarm

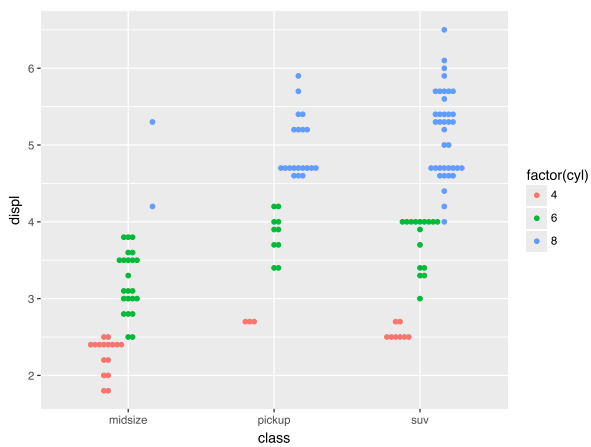
```
(p9.ggplot(sub_mpg, p9.aes("class", "displ", color="factor(cyl)")) +
  geom_beeswarm(dodge_width=.5)).show()
```

```
print(ggplot(sub_mpg, aes(class, displ, color = factor(cyl))) +
  geom_beeswarm(dodge.width = .5))
```

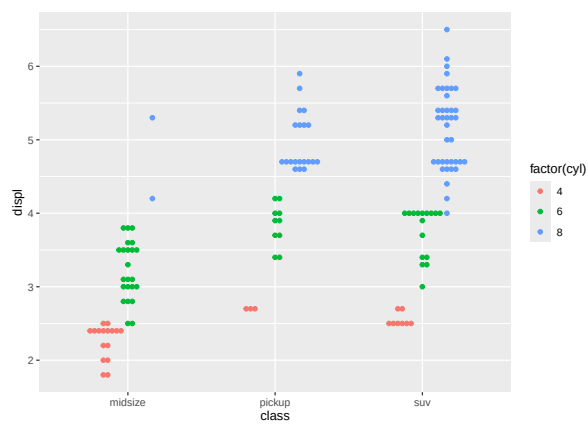
Alternative methods and priority

```
(p9.ggplot(alternative, p9.aes("x", "y")) +
  geom_beeswarm(cex=2.5, method="swarm") + p9.ggtitle('method = "swarm"')).show()
```

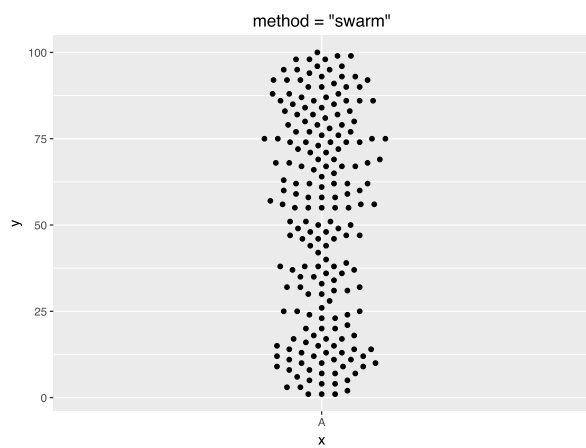
```
print(ggplot(alternative, aes(x, y)) +
  geom_beeswarm(cex = 2.5, method = "swarm") + ggtitle("swarm"))
```



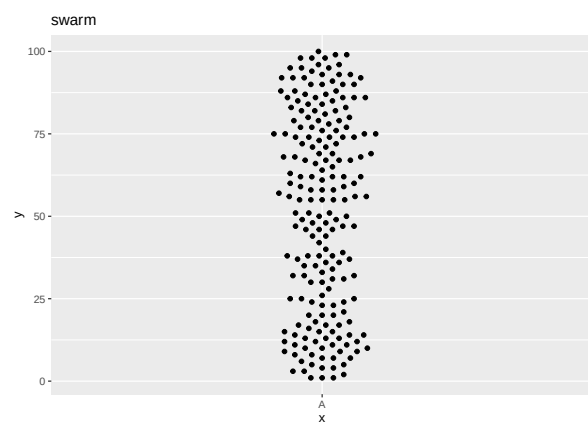
(a) Python: dodged geom_beeswarm



(a) R: dodged geom_beeswarm



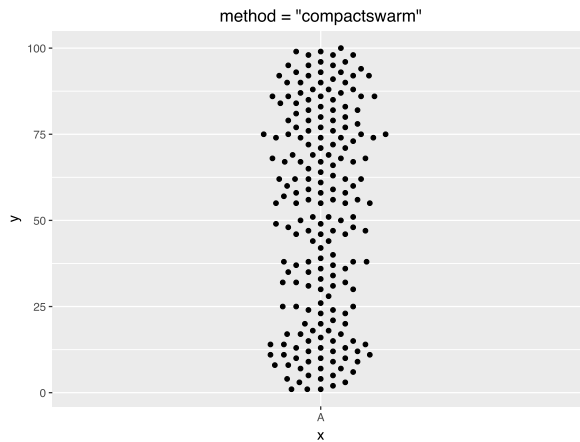
(a) Python: swarm method



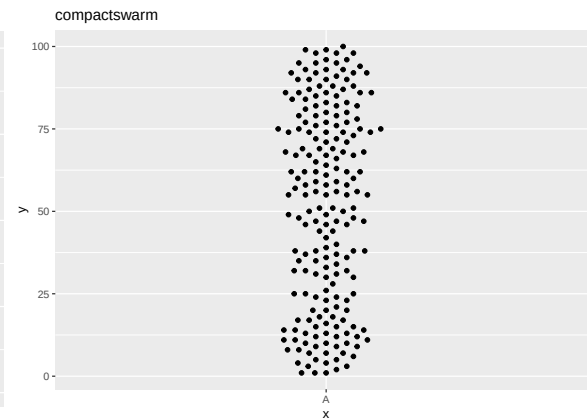
(a) R: swarm method

```
(p9.ggplot(alternative, p9.aes("x", "y")) +
  geom_beeswarm(cex=2.5, method="compactswarm") +
  p9.ggtitle('method = "compactswarm"')).show()
```

```
print(ggplot(alternative, aes(x, y)) +
  geom_beeswarm(cex = 2.5, method = "compactswarm") +
  ggtitle("compactswarm"))
```



(a) Python: compactswarm method



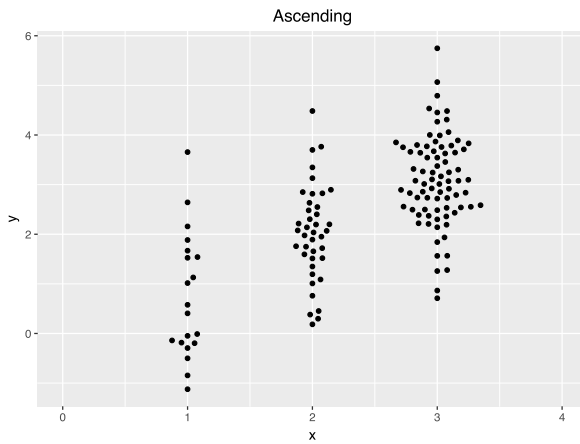
(a) R: compactswarm method

```
(p9.ggplot(priority, p9.aes("x", "y")) +
  geom_beeswarm(cex=2, priority="ascending", random_state=12345) +
  p9.ggtitle("Ascending") +
  p9.scale_x_continuous(limits=(0, 4))).show()
```

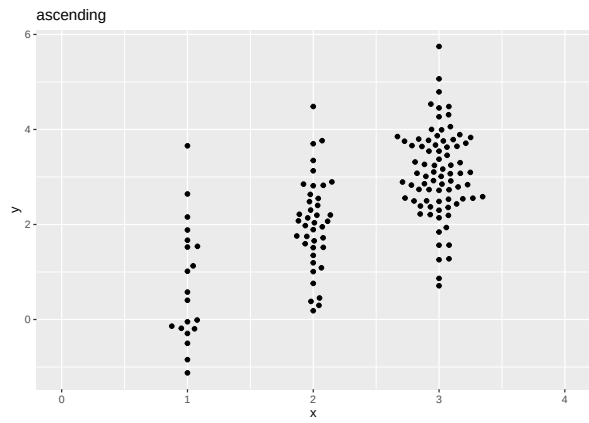
```
print(ggplot(priority, aes(x, y)) +
  geom_beeswarm(cex = 2, priority = "ascending") +
  ggtitle("ascending") +
  scale_x_continuous(limits=c(0, 4)))
```

Corral runaway points

```
(p9.ggplot(corral, p9.aes("id", "y", color="id")) +
  geom_beeswarm(cex=2.5, corral="none", corral_width=.9, random_state=1995) +
  p9.ggtitle('corral = "none"')).show()
```

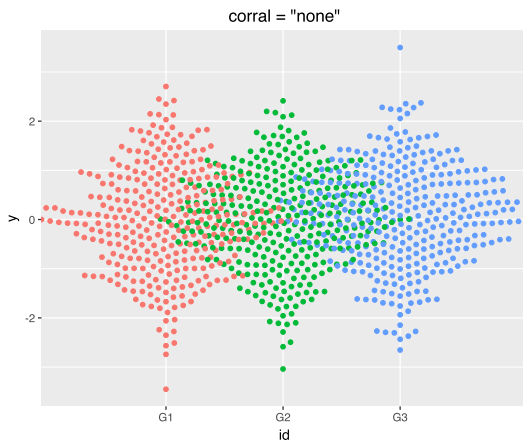


(a) Python: ascending priority

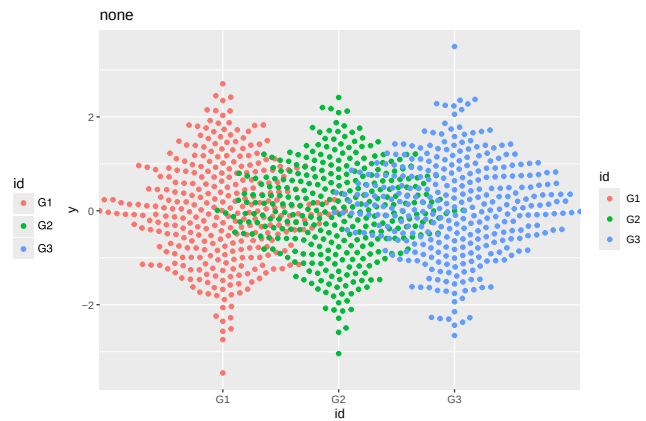


(a) R: ascending priority

```
print(ggplot(corral, aes(id, y, colour = id)) +
      geom_beeswarm(cex = 2.5, corral = "none", corral.width = .9) +
      ggtitle("none"))
```



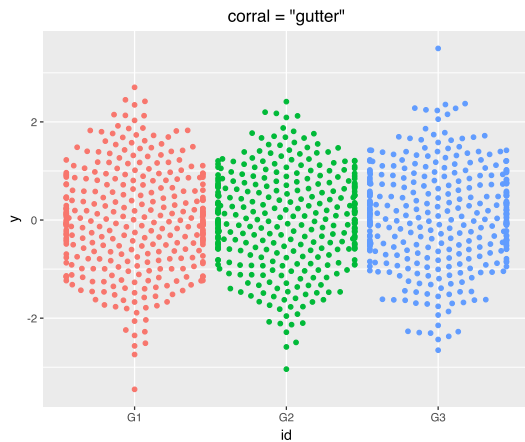
(a) Python: no corraling



(a) R: no corraling

```
(p9.ggplot(corral, p9.aes("id", "y", color="id"))) +
  geom_beeswarm(cex=2.5, corral="gutter", corral_width=.9, random_state=1995) +
  p9.ggtitle('corral = "gutter"]').show()
```

```
print(ggplot(corral, aes(id, y, colour = id)) +
      geom_beeswarm(cex = 2.5, corral = "gutter", corral.width = .9) +
      ggtitle("gutter"))
```



(a) Python: gutter corraling



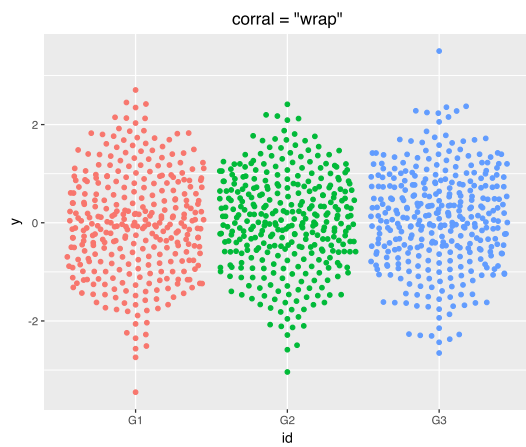
(a) R: gutter corraling

```
(p9.ggplot(corral, p9.aes("id", "y", color="id")) +
  geom_beeswarm(cex=2.5, corral="wrap", corral_width=.9, random_state=1995) +
  p9.ggtitle('corral = "wrap"')).show()
```

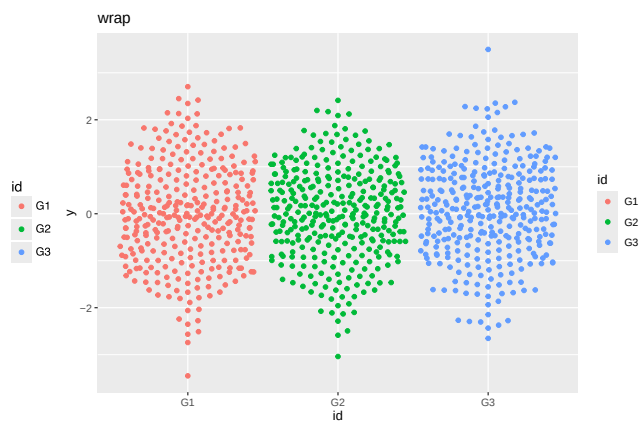
```
print(ggplot(corral, aes(id, y, colour = id)) +
      geom_beeswarm(cex = 2.5, corral = "wrap", corral.width = .9) +
      ggtitle("wrap"))
```

```
(p9.ggplot(corral, p9.aes("id", "y", color="id")) +
  geom_beeswarm(cex=2.5, corral="random", corral_width=.9, random_state=1995) +
  p9.ggtitle('corral = "random"')).show()
```

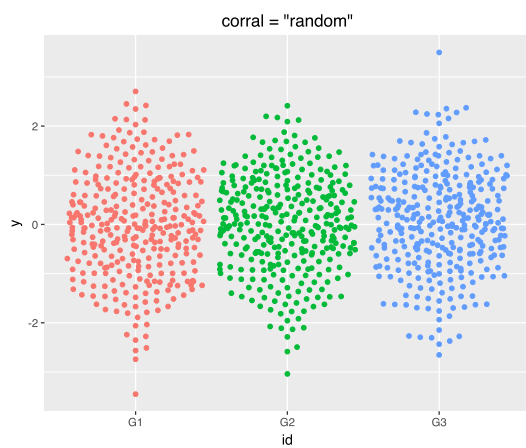
```
print(ggplot(corral, aes(id, y, colour = id)) +
      geom_beeswarm(cex = 2.5, corral = "random", corral.width = .9) +
      ggtitle("random"))
```



(a) Python: wrapped corraling



(a) R: wrapped corraling



(a) Python: random corraling



(a) R: random corraling